



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SE/SS ZG566

Secure Software Engineering

T V Rao



- *The slides presented here are obtained from the authors of the books, product documentations, and from various other contributors. I hereby acknowledge all the contributors for their material and inputs.*
- *I have added and modified slides to suit the requirements of the course.*



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Work products during SDLC

Requirement Models

Scenario-based modeling – represents the system from the user's point of view

Flow-oriented modeling – provides an indication of how data objects are transformed by a set of processing functions

Class-based modeling – defines objects, attributes, and relationships

Behavioral modeling – depicts the states of the classes and the impact of events on these states

SafeHome (from Pressman)

SafeHome: The home security function would protect against and/or recognize a variety of undesirable “situations” such as illegal entry, fire, flooding, carbon monoxide levels, and others. It’ll use our wireless sensors to detect each situation. It can be programmed by the homeowner, and will automatically telephone a monitoring agency when a situation is detected.

Objects described for SafeHome might include the control panel, smoke detectors, window and door sensors, motion detectors, an alarm, an event (a sensor has been activated), a display, a PC, telephone numbers, a telephone call, and so on. The list of services might include configuring the system, setting the alarm, monitoring the sensors, dialing the phone, programming the control panel, and reading the display (note that services act on objects).

Use-Cases

A collection of user scenarios that describe the thread of usage of a system

Each scenario is described from the point-of-view of an “actor”—a person or device that interacts with the software in some way

Each scenario answers the following questions:

- Who is the primary actor, the secondary actor (s)?
- What are the actor’s goals?
- What preconditions should exist before the story begins?
- What main tasks or functions are performed by the actor?
- What extensions might be considered as the story is described?
- What variations in the actor’s interaction are possible?
- What system information will the actor acquire, produce, or change?
- Will the actor have to inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

Use case—detailed example (Pressman)

Example: “SAFEHOME” system (Pressman)

Use case name: *InitiateMonitoring*

Participating actors: homeowner, technicians, sensors

Flow of events (homeowner):

- Homeowner wants to set the system when the homeowner leaves house or remains in house
- Homeowner observes control panel
- Homeowner enters password
- Homeowner selects “stay” or “away”
- Homeowner observes that red alarm light has come on, indicating the system is armed

Use case—detailed example (Pressman)

Pre condition(s)

Homeowner decides to set control panel

Post condition(s)

- Control panel is not ready; homeowner must check all sensors and reset them if necessary
- Control panel indicates incorrect password (one beep)—homeowner enters correct password
- Password not recognized—must contact monitoring and response subsystem to reprogram password
- *Stay* selected: control panel beeps twice and lights *stay* light; perimeter sensors are activated
- *Away* selected: control panel beeps three times and lights *away* light; all sensors are activated

Use case—detailed example (Pressman)

Quality requirements:

- Control panel may display additional text messages
- time the homeowner has to enter the password from the time the first key is pressed
- Ability to activate the system without the use of a password or with an abbreviated password
- Ability to deactivate the system before it actually activates

Misuse case (searchsoftwarequality.techtarget.com)

Name: Attack on "forgot password" functionality

Summary: A malicious user tries to attack the "forgot password" functionality in order to gain access to the Web application or guess a valid e-mail address

Author: Anurag Agarwal

Date: April 15, 2006

Possible Attacks:

- SQL injection attack
- Brute force attack to guess a valid user
- Sniffing attack on e-mail sent with password on an insecure transmission channel

Trigger Point: Can happen anytime

Preconditions: None

Assumptions:

- The attacker can perform this attack remotely over the Internet
- The attacker can be an anonymous user

Misuse case (searchsoftwarequality.techtarget.com)

Worst case threat (post condition) :

- Attacker gains entry into the company database and steals sensitive information
- Attacker is able to modify an existing e-mail address to its own e-mail address and mails the password to himself to gain unauthorized entry into the system

Related business rule:

- The system should e-mail the password to a valid e-mail address entered

Capture guarantee (post condition) :

- Attacker cannot gain access to the database to steal or modify information
- Attacker cannot identify the e-mail address of a valid user
- Attacker cannot view the password sent in an e-mail to an e-mail address of a valid user

Misuse case (searchsoftwarequality.techtarget.com)

Potential misuser profile:

- Script kiddie
- Skilled attacker

Threat level: High

Mitigation steps:

- SQL injection attack
 - List all the mitigation steps to avoid a SQL injection attack.
- Brute force attack
 - Accept first name, last name along with e-mail address
 - Have proper error handling so as not to reveal information to the attacker
 - Delay 3 to 5 seconds before re-entering the e-mail address
 - Lock out page for that IP address after 10 attempts
- Sniffing attack
 - Send password e-mail on a secure transmission channel with strong encryption

Data Flow Modeling

Data Flow Diagram

- Depicts how input is transformed into output as data objects move through a system

Process Specification

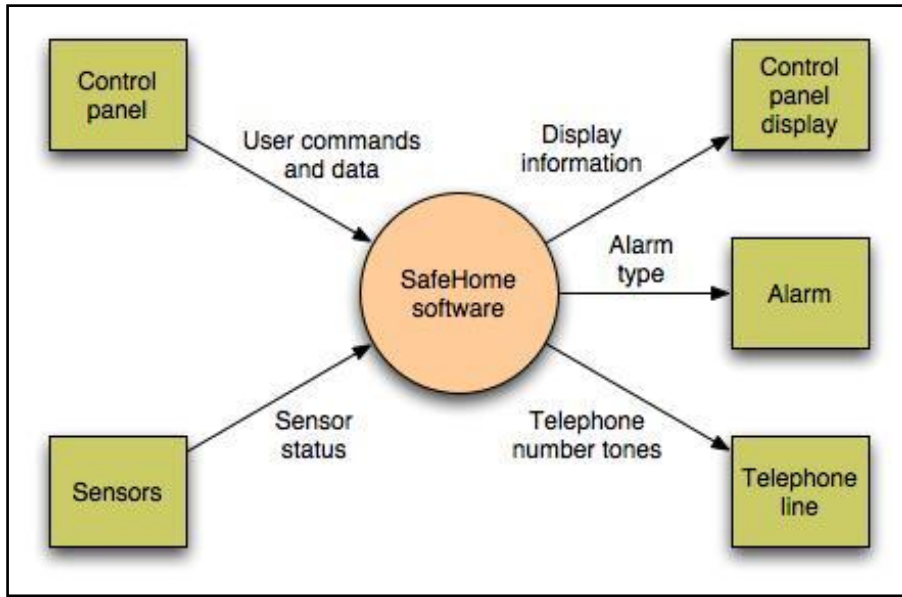
- Describes data flow processing at the lowest level of refinement in the data flow diagrams

Data Flow Modeling

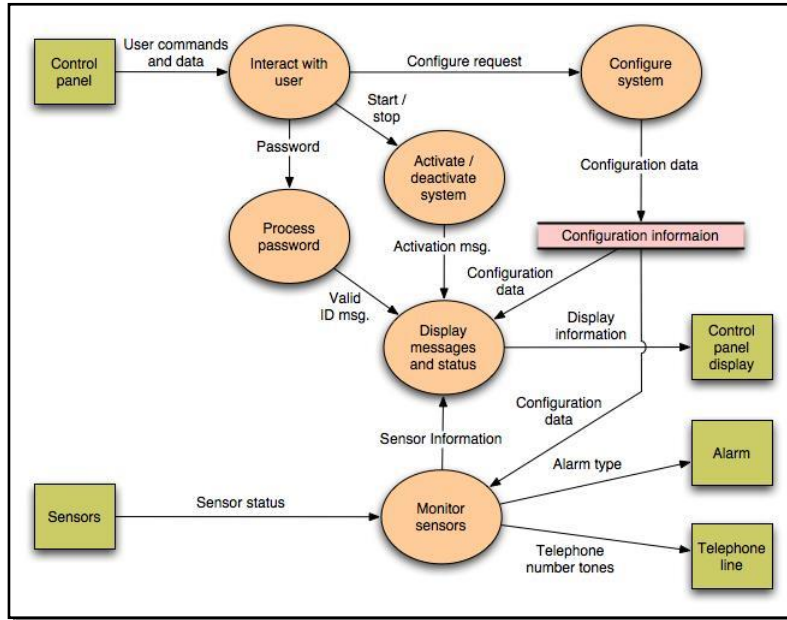
Guidelines

- Depict the system as single bubble in level 0.
- Carefully note primary input and output.
- Refine by isolating candidate processes and their associated data objects and data stores.
- Label all elements with meaningful names.
- Maintain information conformity between levels.
- Refine one bubble at a time.

Data Flow Diagram



Data Flow Diagram (Next Level)





BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Security implications to SDLC

What is OWASP SAMM?

SAMM (Software Assurance Maturity Model) is a OWASP Project

Mission of SAMM is

- To provide an effective and measurable way for all types of organizations to analyze and improve their software security posture.
- To raise awareness and educate organizations on how to design, develop, and deploy secure software through our self-assessment model.
- To support the complete software lifecycle and is technology and process agnostic.

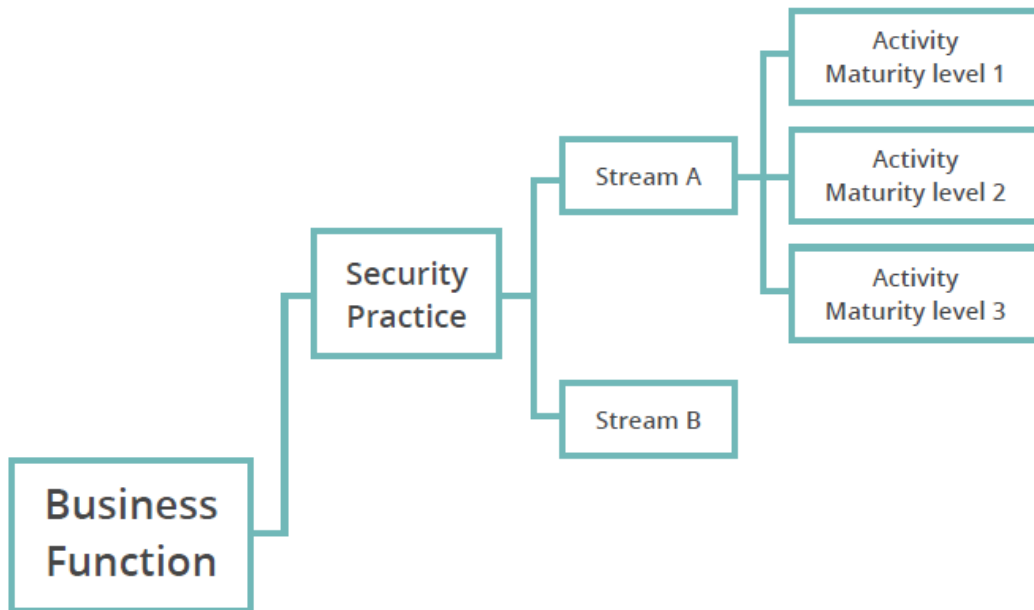
<https://owasp samm.org/model/>

OWASP SAMM

SAMM (Software Assurance Maturity Model) is meant to help organizations assess, formulate, and implement a strategy for software security, that can be integrated into their existing SDLC.

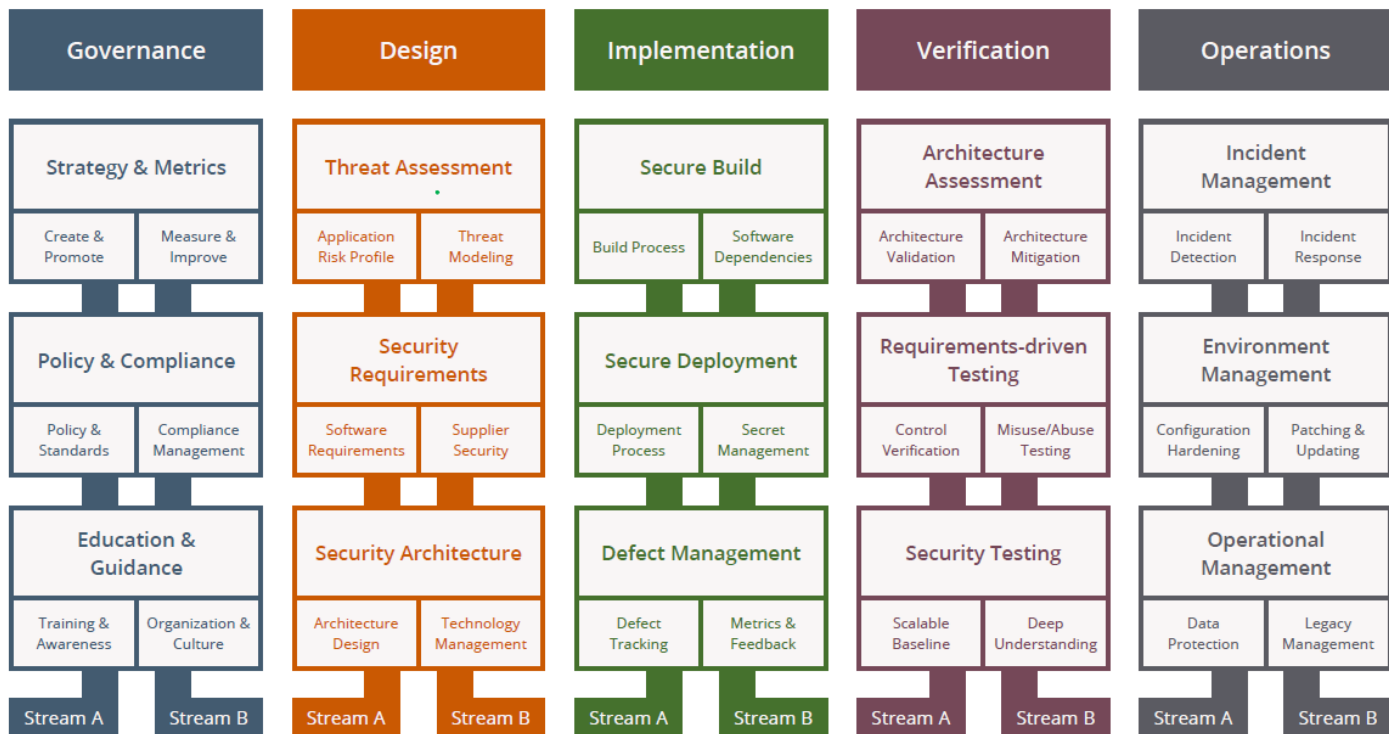
- SAMM is based around 15 security practices grouped into 5 business functions.
- Every security practice contains a set of activities, structured into 3 maturity levels.
- The activities on a lower maturity level are typically easier to execute and require less formalization than the ones on a higher maturity level.

OWASP SAMM



<https://owasp samm.org/model/>

OWASP SAMM Overview



<https://owasp samm.org/model/>

SAMM Checklist

OWASP SAMM Toolkit.xlsx
(<https://owaspsamm.org/assessment/>)

OWASP SAMM Practices

SAMM prescribes methodology for practice that includes

- Assessment
- Results
- Success metrics
- Costs
- Personnel (Roles)
- Levels



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Security implications to SDLC

Security Development Lifecycle

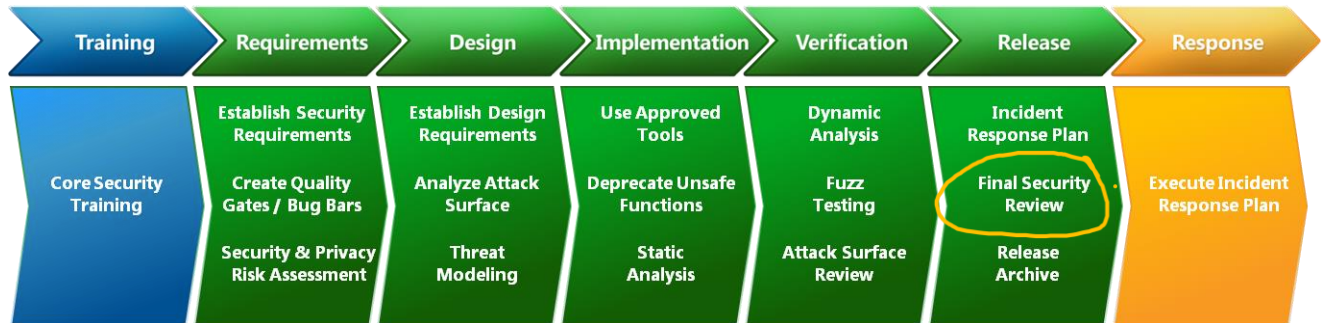
Security Development Lifecycle

The Security Development Lifecycle (SDL) is a security assurance process developed by Microsoft as a company-wide initiative and a mandatory policy to reduce the number and severity of vulnerabilities in software products.

The Microsoft SDL is based on three core concepts —

- education,*
- continuous process improvement, and*
- accountability*

Security Development Lifecycle



SDL - Core Security Training

Basic software security training should cover foundational concepts such as:

- Secure design, including the following topics: Attack surface reduction, Defense in depth, principle of least privilege, Secure defaults
- Threat modeling, including the following topics: Overview of threat modeling, Design implications of a threat model, Coding constraints based on a threat model
- Secure coding, including the following topics: Buffer overruns (for applications using C and C++), Integer arithmetic errors (for applications using C and C++), Cross-site scripting (for managed code and Web applications), SQL injection (for managed code and Web applications), Weak cryptography
- Security testing, including the following topics: Differences between security testing and functional testing, Risk assessment, Security testing methods
- Privacy, including the following topics: Types of privacy-sensitive data, Privacy design best practices, Risk assessment, Privacy development best practices, Privacy testing best practices

SDL Roles

SDL roles are designed to provide project security and privacy oversight and have the authority to accept or reject security and privacy plans from a project team.

Security Advisor/Privacy Advisor. This role is filled by subject-matter experts (SMEs) from outside the project team. The person chosen for this task must fill two sub-roles:

- Auditor: monitors each phase of the software development process and attest to successful completion of each security requirement
- Expert: must possess verifiable subject-matter expertise in security.

Team Champion. should be filled by SMEs from the project team. Responsible for the negotiation, acceptance, and tracking of minimum security and privacy requirements

Some SDL Practices

Threat Modeling

- used in environments where there is meaningful security risk
- allows development teams to consider, document, and discuss the security implications of designs in the context of their planned operational environment
- Threat modeling is a team exercise, encompassing program/project managers, developers, and testers, performed during the software design stage

Attack Surface Reduction

- a means of reducing risk by giving attackers less opportunity to exploit a potential weak spot or vulnerability
- encompasses shutting off or restricting access to system services, applying the principle of least privilege, and employing layered defenses wherever possible

Some SDL Practices

Static Analysis

- The team should be aware of the strengths and weaknesses of static analysis tools and be prepared to augment static analysis tools with other tools or human review as appropriate

Dynamic Program Analysis

- specify tools that monitor application behavior for memory corruption, user privilege issues, and other critical security problems

Fuzz Testing

- a specialized form of dynamic analysis used to induce program failure by deliberately introducing malformed or random data to an application

Some SDL Practices

Final Security Review

- a deliberate examination of all the security activities performed on a software application prior to release
- performed by the security advisor with assistance from the regular development staff and the security and privacy team leads
- includes an examination of threat models, exception requests, tool output, and performance against the previously determined quality gates or bug bars
- If a team does not meet all SDL requirements and the security advisor cannot approve the project, the project cannot be released
- Teams must either address whatever SDL requirements that they can prior to launch or escalate to executive management for a decision



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

BSIMM

The Building Security In Maturity Model (BSIMM)

BSIMM is a study of existing software security initiatives.

Representatives from Cigital® and Fortify® conduct interviews and collect data from companies with sound security practices, including

- Adobe®,
- Dell®,
- EMC®,
- Google™,
- Microsoft®
- etc.

<https://www.synopsys.com/software-integrity/resources/analyst-reports/bsimm.html>

BSIMM Terminology

- **Activity.** Actions or efforts carried out or facilitated by the SSG as part of a practice. Activities are divided into three levels in the BSIMM based on observation rates.
- **Capability.** A set of BSIMM activities spanning one or more practices working together to serve a cohesive security function.
- **Champions.** A group of interested and engaged developers, cloud security engineers, deployment engineers, architects, software managers, testers, or people in similar roles who have an active interest in software security and contribute to the security posture of the organization and its software.
- **Domain.** One of the four categories the framework is divided into, i.e., Governance, Intelligence, SSDL Touchpoints, and Deployment.
- **Participants.** The group of firms in the current data pool.
- **Practice.** A grouping of BSIMM activities. The SSF is organized into 12 practices, three in each of four domains.

BSIMM Terminology

- **Secure SDL (SSDL).** Any software lifecycle with integrated software security checkpoints and activities.
- **Software security framework (SSF).** The basic structure underlying the BSIMM, comprising 12 practices divided into four domains.
- **Software security group (SSG).** The internal group charged with carrying out and facilitating software security. The group's name might also have an appropriate organizational focus, such as application security group or product security group.
- **Software security initiative (SSI).** An organization-wide program to instill, measure, manage, and evolve software security activities in a coordinated fashion. Also referred to in some organizations as an application security program, product security program, or perhaps as a DevSecOps program.

The BSIMM Skeleton

GOVERNANCE

STRATEGY & METRICS

- Publish process and evolve as necessary.
- Educate executives on software security.
- Implement security checkpoints and associated governance.
- Publish data about software security internally and use it to drive change.
- Enforce security checkpoints and track exceptions.
- Create or grow a satellite (security champions).
- Require security sign-off prior to software release.
- Create evangelism role and perform internal marketing.
- Use a software asset tracking application with portfolio view.
- Make SSI efforts part of external marketing.
- Identify metrics and use them to drive resourcing.
- Integrate software-defined lifecycle governance.
- Integrate software supply chain risk management.

COMPLIANCE & POLICY

- Unify regulatory pressures.
- Identify privacy obligations.
- Create policy.
- Build a PII inventory.
- Require security sign-off for compliance-related risk.
- Implement and track controls for compliance.
- Include software security SLAs in all vendor contracts.
- Ensure executive awareness of compliance and privacy obligations.
- Document a software compliance story.
- Ensure compatible vendor policies.
- Drive feedback from software lifecycle data back to policy.

TRAINING

- Conduct software security awareness training.
- Deliver on-demand individual training.
- Include security resources in onboarding.
- Enhance satellite (security champions) through training and events.
- Create and use material specific to company history.
- Deliver role-specific advanced curriculum.
- Host software security events.
- Require an annual refresher.
- Provide expertise via open collaboration channels.
- Reward progression through curriculum.
- Provide training for vendors and outsourced workers.
- Identify new satellite members (security champions) through observation.

The BSIMM Skeleton

INTELLIGENCE		
ATTACK MODELS	SECURITY FEATURES & DESIGN	STANDARDS & REQUIREMENTS
• Use a data classification scheme for software inventory. • Identify potential attackers. • Gather and use attack intelligence. • Build attack patterns and abuse cases tied to potential attackers. • Collect and publish attack stories. • Build an internal forum to discuss attacks. • Have a research group that develops new attack methods. • Monitor automated asset creation. • Create and use automation to mimic attackers. • Create technology-specific attack patterns. • Maintain and use a top N possible attacks list.	• Integrate and deliver security features. • Application architecture teams engage with the SSG. • Leverage secure-by-design components and services. • Create capability to solve difficult design problems. • Form a review board to approve and maintain secure design patterns. • Require use of approved security features and frameworks. • Find and publish secure design patterns from the organization.	• Create security standards. • Create a security portal. • Translate compliance constraints to requirements. • Identify open source. • Create a standards review process. • Create SLA boilerplate. • Control open source risk. • Communicate standards to vendors. • Use secure coding standards. • Create standards for technology stacks.

The BSIMM Skeleton

SSDL TOUCHPOINTS

ARCHITECTURE ANALYSIS

- Perform security feature review.
- Perform design review for high-risk applications.
- Use a risk methodology to rank applications.
- Perform architecture analysis using a defined process.
- Standardize architectural descriptions.
- Have SSG lead design review efforts.
- Have engineering teams lead AA process.
- Drive analysis results into standard design patterns.
- Make the SSG available as an AA resource or mentor.

CODE REVIEW

- Perform opportunistic code review.
- Use automated code review tools.
- Make code review mandatory for all projects.
- Assign code review tool mentors.
- Use custom rules with automated code review tools.
- Use a top N bugs list (real data preferred).
- Use centralized defect reporting to close the knowledge loop.
- Build a capability to combine AST results.
- Create capability to eradicate bugs.
- Automate malicious code detection.
- Enforce secure coding standards.

SECURITY TESTING

- Perform edge/boundary value condition testing during QA.
- Drive tests with security requirements and security features.
- Integrate opaque-box security tools into the QA process.
- Drive QA tests with AST results.
- Include security tests in QA automation.
- Perform fuzz testing customized to application APIs.
- Drive tests with design review results.
- Leverage code coverage analysis.
- Begin to build and apply adversarial security tests (abuse cases).
- Implement event-driven security testing in automation.

The BSIMM Skeleton

DEPLOYMENT

PENETRATION TESTING

- Use external penetration testers to find problems.
- Feed results to the defect management and mitigation system.
- Use penetration testing tools internally.
- Penetration testers use all available information.
- Schedule periodic penetration tests for application coverage.
- Use external penetration testers to perform deep-dive analysis.
- Customize penetration testing tools.

SOFTWARE ENVIRONMENT

- Use application input monitoring.
- Ensure host and network security basics are in place.
- Implement cloud security controls.
- Define secure deployment parameters and configurations.
- Protect code integrity.
- Use application containers to support security goals.
- Use orchestration for containers and virtualized environments.
- Use code protection.
- Use application behavior monitoring and diagnostics.
- Create bills of materials for deployed software.
- Perform application composition analysis on code repositories.
- Protect integrity of development toolchains.

CONFIGURATION MANAGEMENT & VULNERABILITY MANAGEMENT

- Create or interface with incident response.
- Identify software defects found in operations monitoring and feed them back to engineering.
- Track software defects found in operations through the fix process.
- Have emergency response.
- Develop an operations software inventory.
- Fix all occurrences of software defects found in operations.
- Enhance the SSDL to prevent software defects found in operations.
- Simulate software crises.
- Operate a bug bounty program.
- Automate verification of operational infrastructure security.
- Publish risk data for deployable artifacts.
- Streamline incoming responsible vulnerability disclosure.
- Do attack surface management for deployed applications.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Security in Agile Development

The Manifesto for Agile Software Development

“We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.”

Kent Beck et al



What is “Agility”?

Effective (rapid and adaptive) response to change

Effective communication among all stakeholders

Drawing the customer onto the team

Organizing a team so that it is in control of the work performed

Yielding ...

Rapid, incremental delivery of software

Table 1: Security Activities for Agile Software Development

Security Activity	Definition
<i>Addressing security in early iterations with requirements and testing</i>	This security activity emphasizes the importance of development teams addressing security issues and concerns early in the project before deploying the software.
<i>Stating security requirements that are expected in the production software</i>	This requires incorporating security expectations in project requirements when describing the responsibilities and behavior of the software.
<i>Adding a security specialist to your team</i>	Security specialists, such as a Security Master, are members of a development team that focus on security aspects of the project to address concerns and ensure the security of the system.
<i>Additional points or weights to issues with an impact on security</i>	This activity involves increasing the weights, such as story points in an Agile development environment, of issues that will have a higher impact the security of the product to prioritize security-related tasks and encourage more secure development and testing.
<i>Iterative and incremental vulnerability and penetration testing</i>	This security activity suggests incorporating recurring security scanning, such as Dynamic Application Security Testing (DAST), to test for security flaws in the working software automatically.
<i>Iterative and incremental security static analysis</i>	Similar to DAST, Static Application Security Testing (SAST) involves using security-related static analysis tools to detect potential security vulnerabilities by scanning the source code.
<i>Iterative and incremental risk analysis, countermeasure graphs</i>	This security activity consists of using tools to monitor networks, applications, and infrastructure and perform risk analysis to identify vulnerabilities. These tools can evaluate the system's security and suggest methods to prevent attacks.
<i>Automatic testing</i>	This security activity involves incorporating secure coding practices, such as vulnerabilities analysis and risk assessment, into the deployment pipeline for software projects. This allows security checks to be automatically triggered with code changes and issues to be addressed before the software is deployed to users.

Security in Agile Development

1. Utilize the hacker in that developer

A developer with the right tools, training and mindset can uncover the most common security pitfalls in code.

—Developers must be empowered through training and the provision of the right tools that will enable them to analyze code before submitting it to quality assurance or testing teams.

All development team members must be aware of the most common vulnerabilities in each system they develop.

—With developers' ratio to security specialists standing at about 100:1, it is only plausible to pass the responsibility of securing systems to the developers. They can then perform routine scans of their code and fix any issues that may arise during the core development process.

Team leaders can decide to incorporate the security team members into individual development teams to discover vulnerabilities early in development.

2. Always consider the “evil” user interacting with the system

Many developers envision a perfect software without taking into account the possibility of an “evil” user interaction

Team players should be made aware of the possible ways an attacker may interact with the system.

This is a continuous process starting with the statement of security-based user stories.

Only a few clients will talk about security in user stories. Security team has to identify critical points and valuable assets of the software that may be targeted by an attacker. They can enumerate possible attack goals and advise the development team in advance.

Security in Agile Development

3. Uphold continuous integration practices, tools and platforms

There are tools to integrate with development platforms and give insightful information about code.

These include code integration tools, code scanners and shared repositories.

Code analysis tools are available to help debug and refactor buggy code, thereby improving the system's overall safety.

Upholding continuous integration best practices work and help improve the quality of the software, thereby assuring the security of users, user data and vendors.

4. Review user stories with every iteration and adapt as necessary

Agile methodologies have short iterations, resulting in the release of new or improved software system features.

—Therefore, it is possible to review all user stories that necessitate iteration and how they relate to the system's general security before releasing it for public use.

The team leader must also organize peer review of code and how frequently this occurs. Two pairs of eyes are often better than one during code review.

The software must be adapted to better its reliability and security if vulnerabilities are discovered in the review process.

Security in Agile Development

5. Innovate with security

Agile teams neglect security due to absence of robust, agile security practices and testing tools.

Progressive teams have to innovate workable security policies and developing tools based on such policies.

Most CI tools are open to customization.

Therefore, an organization or team can choose a tool that works with their development practices and customize it to serve the most common security concerns affecting their code.

6. Cultivate the culture of security

Organizational culture influences how things are generally done.

If team leaders have high regard for secure systems, the juniors are likely to copy the same and practice it when writing code.

Educating the team and organizing seminars for security and agile development can also help members adopt the security needs of software quickly.

Every new team member should be taken through the necessary software security procedures and guidelines of organization.

Furthermore, combining all the other security aspects of agile development with a supportive and dynamic security culture will boost the general feel of your software.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

DevOps and DevSecOps

DevOps

A set of practices that combines software development (Dev) and IT operations (Ops)



Why DevOps? Iterative and Rapid Software release cycles mean software moves between development team to IT operations team repeatedly. It means strong binding is needed between two teams.

DevOps is complementary with Agile software development; several DevOps aspects came from the Agile methodology

DevOps Practices

Continuous integration and continuous delivery (CI/CD)

- CI - merge code changes frequently into the main code. CD - frequent, automated deployment of new versions into a production environment

Version Control

- tracking revisions and change history to make code easy to review and recover

Agile software development

- emphasizes team collaboration, customer and user feedback, and rapid change through short release cycles

Infrastructure as code

- defines system resources and topologies in a descriptive manner that allows teams to manage those resources as they would code

Configuration management

- managing the state of resources in a system including servers, virtual machines, and databases

Continuous monitoring

- having real-time visibility into the performance and health of the entire stack, from the underlying infrastructure to higher-level software components

What is DevSecOps?

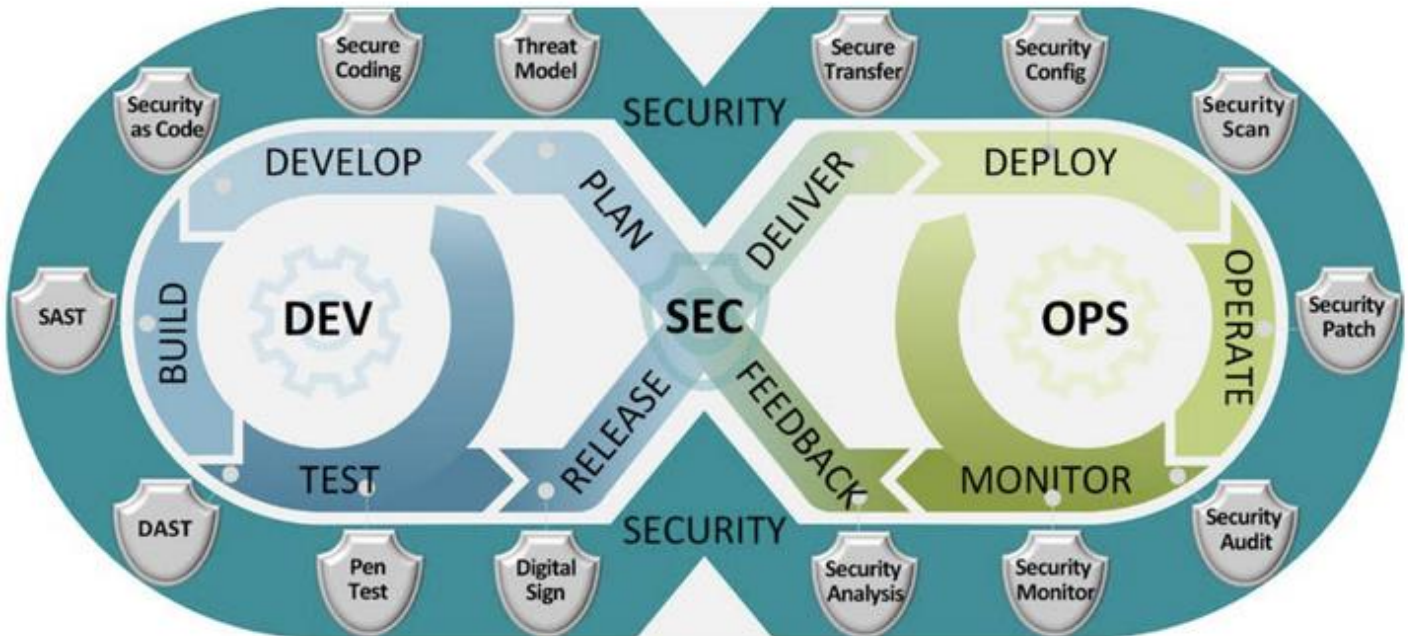
DevSecOps—short for *development, security, and operations*—automates the integration of security at every phase of the software development lifecycle, from initial design through integration, testing, deployment, and software delivery.

- In the past, security was incorporated to software at the end of the development cycle by a separate security team and was tested by a separate QA team

DevSecOps integrates application and infrastructure security seamlessly into Agile and DevOps processes and tools.

DevSecOps makes application and infrastructure security a shared responsibility of development, security, and IT operations teams, rather than the sole responsibility of a security silo

DevSecOps Software Lifecycle



Best Practices for DevSecOps

Shift left

- It encourages software engineers to move security from the right (end) to the left (beginning) of the DevOps (delivery) process.
- Shifting left allows the DevSecOps team to identify security risks and exposures early and ensures that these security threats are addressed immediately

Security education

- development engineers, operations teams, and compliance teams should be familiar with the basic principles of application security and other security engineering practices

Traceability, auditability, and visibility

- Traceability helps to track configuration items across the development cycle to where requirements are implemented in the code
- Auditability is for ensuring compliance with security controls
- Visibility means the organization has a measure the heartbeat of the operation, send alerts, has awareness of cyberattacks as they occur, and provide accountability

Why DevSecOps is Not Easy?

<https://insights.sei.cmu.edu/blog/5-challenges-to-implementing-devsecops-and-how-to-overcome-them/>

The most common challenges are:

- Lack of security assurance at the business and project levels
- Organizational barriers related to collaboration, tooling, and culture
- Impact to quality because security is not a priority while systems are getting more complex
- Lack of security skills for developers, business stakeholders, and auditors
- Insufficient security guidance due to lack of resources, standards, and data

Shift Left

Shifting Left in Software Engineering

<https://dzone.com/articles/shifting-left-in-software-engineering>

<https://solutionshub.epam.com/blog/post/shift-left-testing>

<https://www.oshyn.com/blog/shift-left-test-automation>

Low Code Platform Security

<https://owasp.org/www-project-top-10-low-code-no-code-security-risks/>

<https://www.outsystems.com/blog/posts/low-code-security/>



Software Engineering: A Practitioner's Approach, 7/e Roger Pressman

Software Engineering, Pearson Education, 9th Ed., 2010. Ian Sommerville

SWEBOK by IEEE/ACM

www.owasp.com

sei.cmu.edu

www.microsoft.com

www.devopedia.org/devops

www.ibm.com



Thank You!